

10.3.5 多线程推理与视觉跟随控制

课程信息

- 环境：Windows 10、WSL2 Ubuntu 24.04、ROS 2 Jazzy；推荐 Gazebo Harmonic 做「红柱绕圈跟随」演示；**不强制**安装 PyTorch / ultralytics。
- **认知主线**：采集/推理如何解耦 → 确认事件如何变成速度 → 谁抢 `/cmd_vel` → 丢失/急停如何安全收场。
- **跟踪原理定位**：本节用的是归一化坐标上的比例控制（P）——够课堂推导、够单测、够演示；**不是完整** PID / 卡尔曼 / DeepSORT。
- 动手目标：lite+synthetic 零依赖跑通 → 使能/丢失/遥控抢占/急停 → Gazebo 红柱绕圈；**CP5** 验收。
- 主入口：`follow.launch.py`。
- **源码走读**（对着仓库读注释）：`docs/10.3.5_工程版源码走读.md`（本地 Markdown；课堂白板见同名 `.excalidraw`）。

路径约定：项目位于 `~/mecamind_ros2`。前置：第四节 `scripts/test_cp4.sh` 能过。

讲到具体实现时，正文会用「源码：…」标出文件路径与符号名；请开着编辑器对照。关键文件顶部已有教学注释。

0. 源码地图（整节课对照表）

讲义块	源码位置	必读符号 / 注释入口
多线程 最新帧	src/mecamind_perception/mecamind_perception/detector_node.py	文件头 docstring; LatestFrameQueue; put_latest; _inference_loop
连续帧 确认	src/mecamind_tools/mecamind_tools/perception_filter.py	文件头; select_detection; PerceptionFilterNode._cb
P 跟随 控制律	src/mecamind_tools/mecamind_tools/vision_follow_controller.py	文件头; compute_follow_command; _watchdog; _enable_cb
速度仲裁	src/mecamind_perception/mecamind_perception/cmd_vel_arbiter.py	文件头; pick_active_source; _estop_cb; _tick
安全门	src/mecamind_tools/mecamind_tools/safety_layer.py（入口名 mecamind_safety_gate）	文件头; analyze_scan; gate_twist
红柱绕 圈	src/mecamind_tools/mecamind_tools/follow_target_mover.py	文件头; _on_timer; _set_pose; _follow_enable_cb
跟随世界	src/mecamind_gazebo/worlds/three_room_house_follow.sdf	mecamind_mecanum / follow_target 的 <pose>
总装 launch	src/mecamind_bringup/launch/follow.launch.py	文件头链路图; generate_launch_description
CP5	scripts/test_cp5.sh	文件头验收点列表

更细的逐文件串讲见 《10.3.5_工程版源码走读》。

先读这段：一张总框架图 + 四个问题

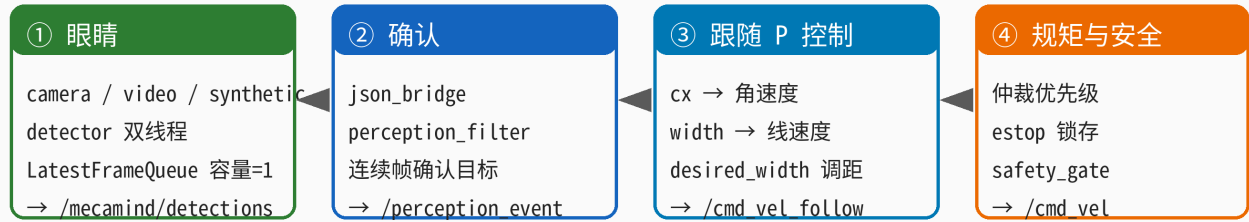
前四课机器人有了「腿」和「眼睛」。第五节要把「看见」变成安全、可抢占、可关掉的「跟着走」。

很多人一上来就拧 `kp_linear`，结果卡在：推理堆旧帧、多模块抢 `/cmd_vel`、丢目标还在冲。

本课反过来：先立规矩，再谈增益。

- ① 采集与推理为何解耦？ → LatestFrameQueue（容量 1）+ 双线程
- ② 确认事件如何变速度？ → `cx / width` → P 控制 → `/cmd_vel_follow`
- ③ 谁抢 `/cmd_vel`？ → 仲裁：estop > teleop > follow > nav
- ④ 丢失/急停如何安全？ → 看门狗、follow_enable、estop 锁存 + 安全门

10.3.5 视觉跟随总框架（课堂版）



速度竞争怎么破？仲裁优先级（左高右低）



新鲜度超时：某路沉默后自动让位；急停锁存：必须显式发 false 才解除

本课跟踪原理 = 归一化坐标上的比例控制（P），不是卡尔曼 / DeepSORT / 完整 PID

第四节把「眼睛」接进 ROS；第五节把「看见」变成「规矩地跟着走」。

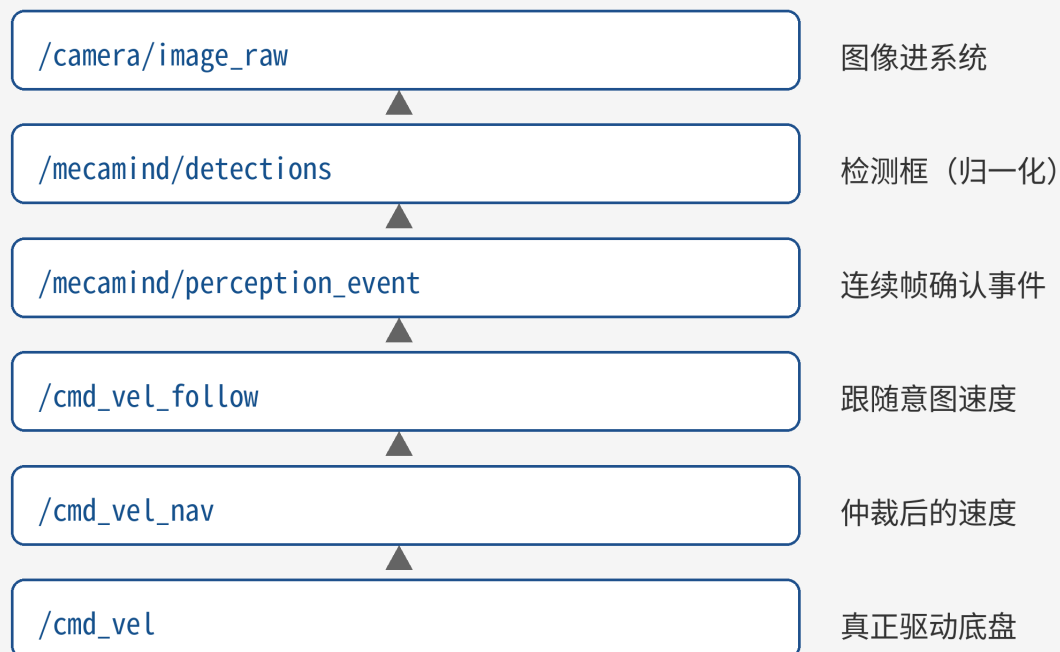
第五节总框架：眼睛 → 确认 → P 跟随 → 仲裁与安全

口诀：

先 lite 合成红块验证方向，再 Gazebo 红柱绕圈看效果；永远先开 follow_enable，永远能用 estop 刹停。

数据流也可以竖着记：

话题数据流（跟随时建议 echo 的）



跟随时建议 echo 的话题链

1. 本节学习目标

1. 用自己的话讲清跟随四问，并能指着总框架图说出每一块对应哪个节点/话题。
2. 说明本课跟踪原理是什么、**故意不引入**哪些更复杂方法、为什么。
3. 解释 LatestFrameQueue 容量为 1 与 dropped_frames 的含义。
4. 说清 desired_width 如何调近/调远；能演示遥控抢占与急停锁存。
5. 跑通 LAB1~4，并用 scripts/test_cp5.sh 得到 MECAMIND_CP5_OK。

2. 主线详解

2.1 主线①：采集与推理为何必须解耦？

相机 15~30 Hz，CPU 上 YOLO 可能要几百毫秒一帧。若同步阻塞或无界堆帧，车会对着「半秒前的幽灵」转向。

正确做法：采集线程只塞「当前最新画面」；推理线程慢慢算。槽位容量 = 1，旧帧直接丢。

源码： src/mecamind_perception/mecamind_perception/detector_node.py

- 类 LatestFrameQueue (约第 79 行起)
 - 方法 put_latest: 队列满则丢旧帧, dropped += 1
 - 方法 _inference_loop: 推理线程主循环
- 文件顶部 docstring 有完整流水线说明, 建议先读注释再读实现。

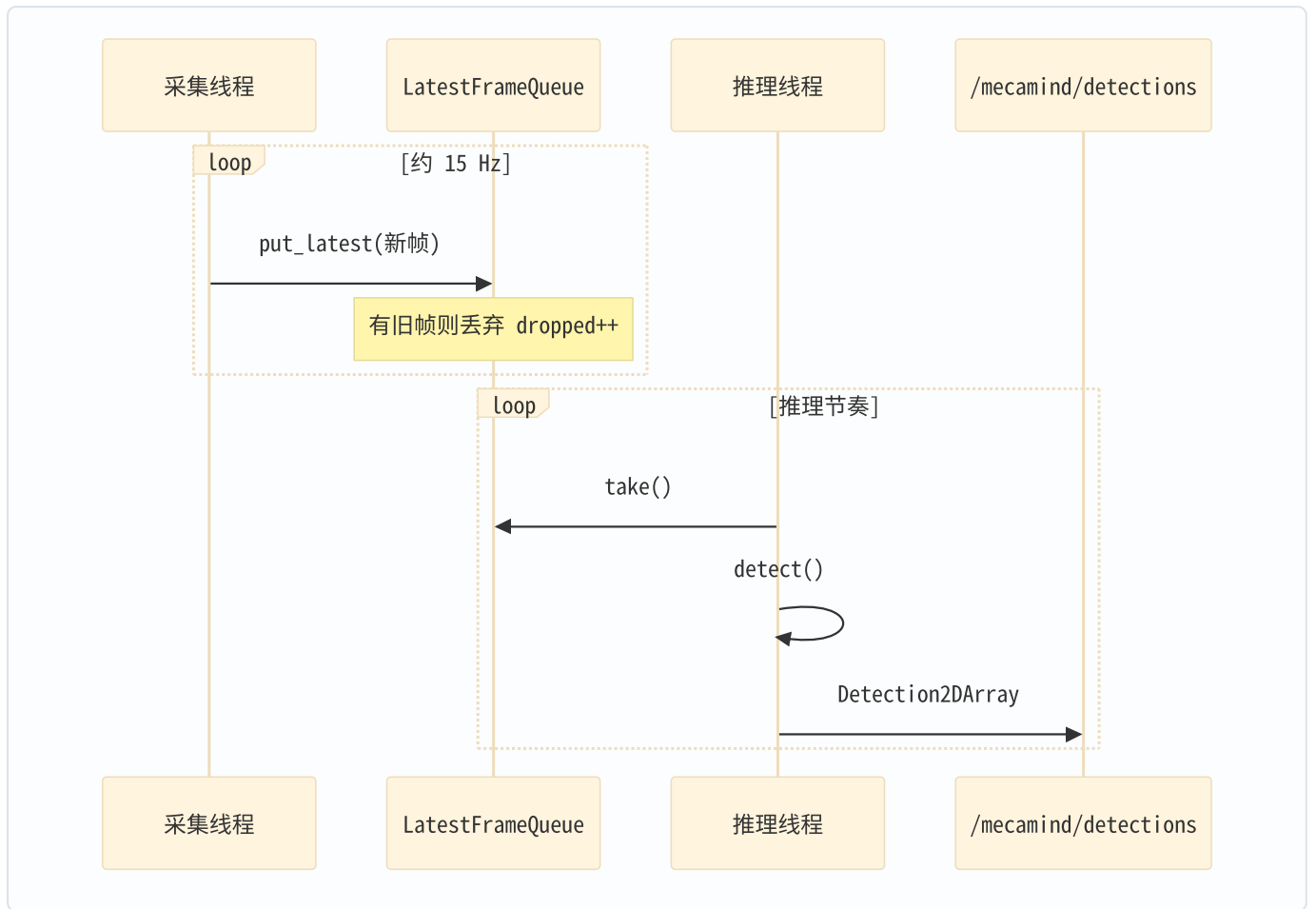
多线程推理: 为何队列容量 = 1?



dropped_frames 上涨通常是健康信号: 系统在用「丢旧帧」换「低延迟」。

多线程 + 最新帧队列 (容量 1)

做法	后果
同步阻塞回调	执行器卡住, 像「眼睛在眨眼」
无界队列堆帧	永远在消化旧画面
容量 1 最新帧	延迟可控, dropped_frames 上涨通常健康



代码落点：mecamind_perception/detector_node.py 中的 LatestFrameQueue、_capture_loop、_inference_loop。

2.2 主线②：跟踪原理——我们用什么、不用什么

2.2.1 先把话说死：本节不是「高级视觉跟踪课」

工业上常见完整 PID、卡尔曼平滑、DeepSORT 多目标 ID、深度估计、视觉伺服雅可比、MPC.....

第五节目标不是把这些一次讲完，而是先交付一条可演示、可验收的链路：

看见目标 → 用简单控制律算出速度意图 → 经仲裁与安全门真正开车。

跟踪原理：课堂用什么 / 故意不用什么

本课实际使用（够用就好）

- 1. 目标选择：label + 连续帧确认
- 2. 水平误差： $e = cx - 0.5$
- 3. 角速度： $\omega = -k_p \cdot e$ （纯比例）
- 4. 远近： $p = width / desired_width$
- 5. $p < 1$ 前进, $p \approx 1$ 跟住, $p > 1$ 后退
- 6. 丢目标：看门狗发零速

优点：可手推、可单测、课堂好讲
输入：只要有 Detection2D 即可

本课故意不用（后续可扩展）

完整 PID（积分 / 微分项）
卡尔曼滤波平滑轨迹
DeepSORT / ByteTrack 多目标 ID
单目深度估计 / 双目测距
完整视觉伺服雅可比
模型预测控制 MPC

不是它们不好，
而是第五节目标是先跑通：
「看见 → 规矩地跟着走」。

课堂用的简单原理 vs 故意不用的复杂方法

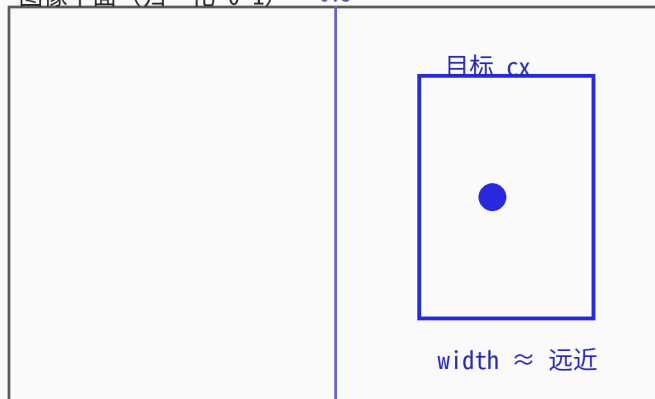
2.2.2 本课实际控制律：归一化坐标上的 P 控制

过滤器确认后，跟随控制器只看两个数：

字段	含义	用途
cx	框中心（0~1）	转向
width	框宽（0~1）	远近

跟随控制律（教学版 P 控制）

图像平面（归一化 0~1） 0.5



公式速记

```
e = cx - 0.5
ω = -kp_ang · e
p = width / desired_w
p < 1: v = vmax · (1-p)
p > 1: v = -vmax · backoff
```

error = cx - 0.5 > 0 ⇒ 右转 angular.z < 0

desired_width 越大 → 目标要「看来越大」才停 → 跟得越近

p < 1 前进 | p ≈ 1 跟随 | p > 1 后退（防穿模）

控制律板书：误差、角速度、远近

源码： src/mecamind_tools/mecamind_tools/vision_follow_controller.py

- 纯函数 compute_follow_command（约第 56 行）——控制律本体，可单测
 - should_publish_follow ——使能 ∧ confirmed 双闸门
 - _event_cb / _watchdog / _enable_cb ——事件驱动 + 超时刹停 + 关跟随主动零速
- 打开文件后请先读顶部「三层保护」注释，再对照下面公式看函数体。

手推一遍：

水平误差：

```
e = cx - 0.5
|e| < deadband → 不转（防抖）
否则 ω = -kp_angular · e
（图像偏右 e>0 → ROS 右转 angular.z<0）
```

远近（用宽度近似，没有深度相机）：

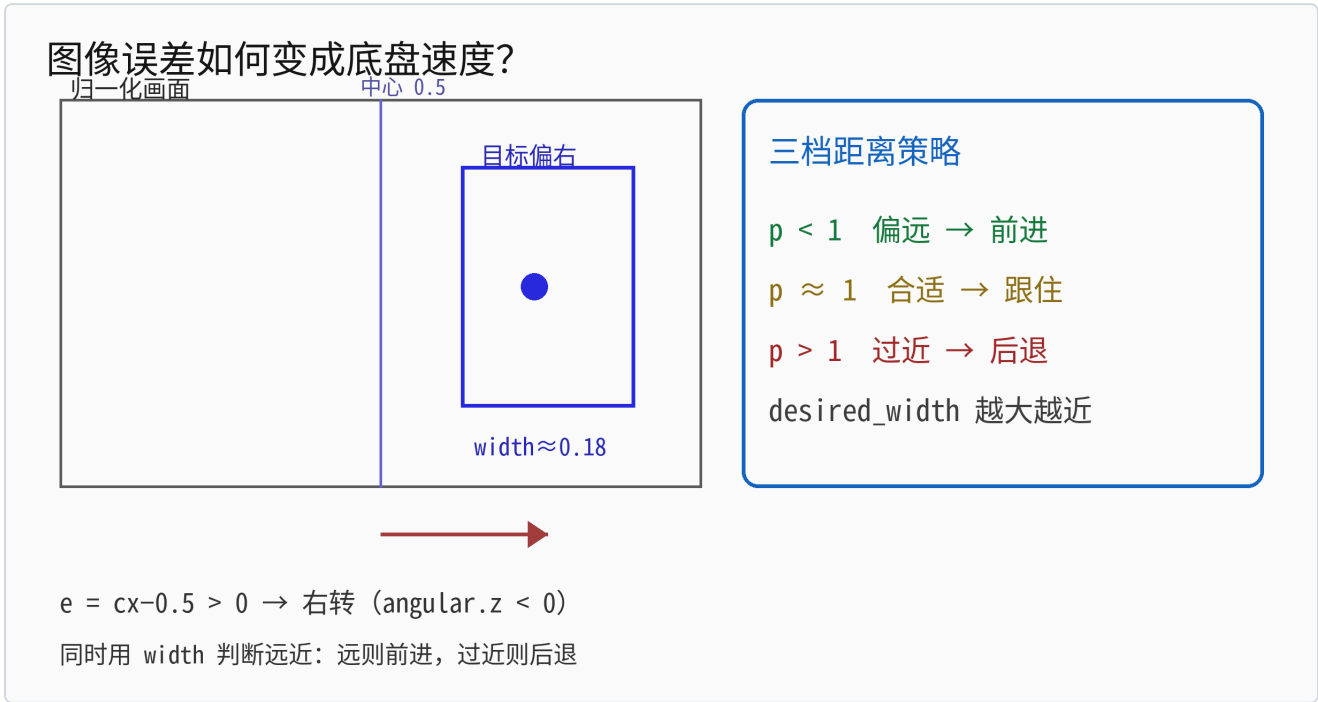
```
p = width / desired_width
p < 1 → 前进（越远越快，上限 max_linear）
p ≈ 1 → 跟住
p > 1 → 后退（本课加大后退，减轻顶柱/穿模）
```

desired_width 是跟近/跟远的主旋钮（越大越近）：

取值	直觉
0.28 左右	跟得远一点，更安全
0.36 (当前演示)	再近约 1/3，课堂观感更好
0.42+	很近，容易贴脸，需强后退

口诀：**desired_width 越大，跟得越近。** 先拧它，再微调 kp_* 。

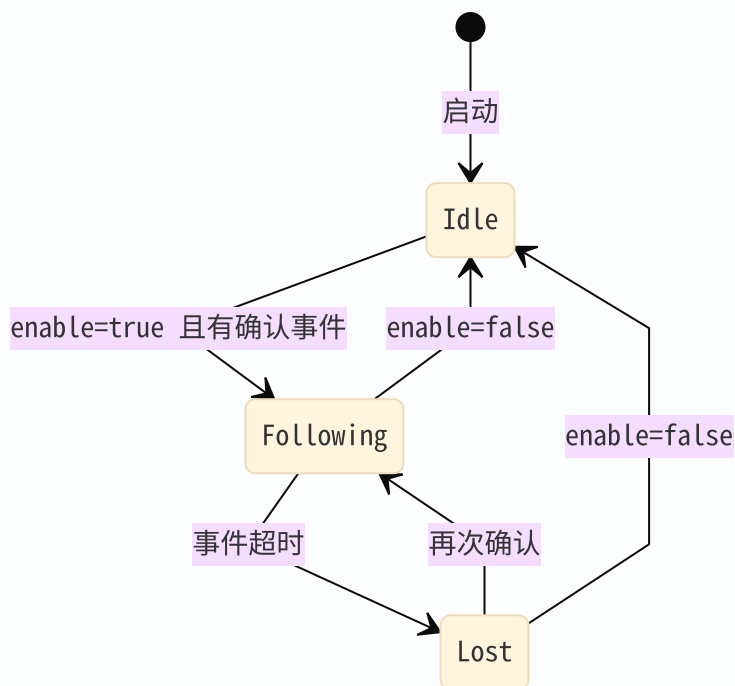
再看一张「目标偏右该怎么转」的示意：



目标偏右：error > 0 $\rightarrow$ 右转

2.2.3 使能开关与丢失看门狗

1. `/mecamind/follow_enable`：未使能不发有效跟随速度；launch 约 10 s 后自动连发打开。
2. 事件超时看门狗：收不到 confirmed $\rightarrow$ 零速（可选 `search_on_loss` 慢搜）。



速度发到 `/cmd_vel_follow`，不直接写 `/cmd_vel` —— 「我想怎么跟」和「系统允不允许」拆开。

把「看 → 算 → 意图 → 规矩 → 动 → 再看」画成闭环，避免和「多目标跟踪器」混淆：

跟随闭环（本课真实在跑的）



没有「跨帧目标 ID」：每一拍都重新检测 + 重新算速度。

这就是为什么叫「检测框伺服的极简版」，而不是 DeepSORT 那种跟踪器。

复杂跟踪器可以后加；本课先保证：看得见、算得清、抢得过、刹得住。

跟随闭环：检测框伺服，而不是 DeepSORT

2.3 主线③：谁抢 `/cmd_vel`?

速度仲裁器：谁说了算？

0	ESTOP 急停	/mecamind/estop 锁存 → 恒为零速
1	TELEOP 遥控	/cmd_vel_teleop 人永远能抢过机器
2	FOLLOW 跟随	/cmd_vel_follow 视觉跟随意图
3	NAV 导航	/cmd_vel_nav_in Nav2（本课可不启）

仲裁优先级：急停 > 遥控 > 跟随 > 导航

源码: [src/mecamind_perception/mecamind_perception/cmd_vel_arbiter.py](#)

- `SOURCE_PRIORITY = ("teleop", "follow", "nav")`
- `pick_active_source` ——按优先级 + 新鲜度选获胜源（纯函数）
- `estop_cb` ——急停**锁存**； `_tick` ——抢占刹车与定时输出

安全门在下游: `src/mecamind_tools/mecamind_tools/safety_layer.py` (`mecamind_safety_gate`)。

机制	含义
优先级	teleop > follow > nav; estop 最高
新鲜度超时	某路沉默则让位
抢占刹车	切换源时先发一帧零速
急停锁存	True 后保持，必须显式 False 解除

```

遥控 —— /cmd_vel_teleop ——|
跟随 —— /cmd_vel_follow ——|   estop > teleop > follow > nav
Nav2 — /cmd_vel_nav_in ——|   └─ arbiter → /cmd_vel_nav → safety_gate → /cmd_vel

```

仲裁管「听谁的」，安全门管「听了之后撞不撞墙」。

运行时可看：

```
ros2 topic echo /mecamind/arbiter_state
```

运行时观察：/mecamind/arbiter_state

典型字段示例：

```
{ "active": "follow", "estop": false }
```

遥控介入时 active 变为 teleop；急停时输出恒为零。

用 `ros2 topic echo /mecamind/arbiter_state` 自己看切换。

arbiter_state 运行截取示例

2.4 主线④：丢失与急停

坏情况	期望	谁负责
目标出画/不再确认	零速或慢搜	跟随看门狗
<code>follow_enable=false</code>	跟随不再开车	双条件闸门
人手动遥控	立刻听人	仲裁
<code>estop=true</code>	恒零直到解除	仲裁锁存
激光过近	减速/刹停	安全门

```
ros2 topic pub --once /mecamind/estop std_msgs/msg/Bool '{data: true}'
ros2 topic pub --once /mecamind/estop std_msgs/msg/Bool '{data: false}'
```

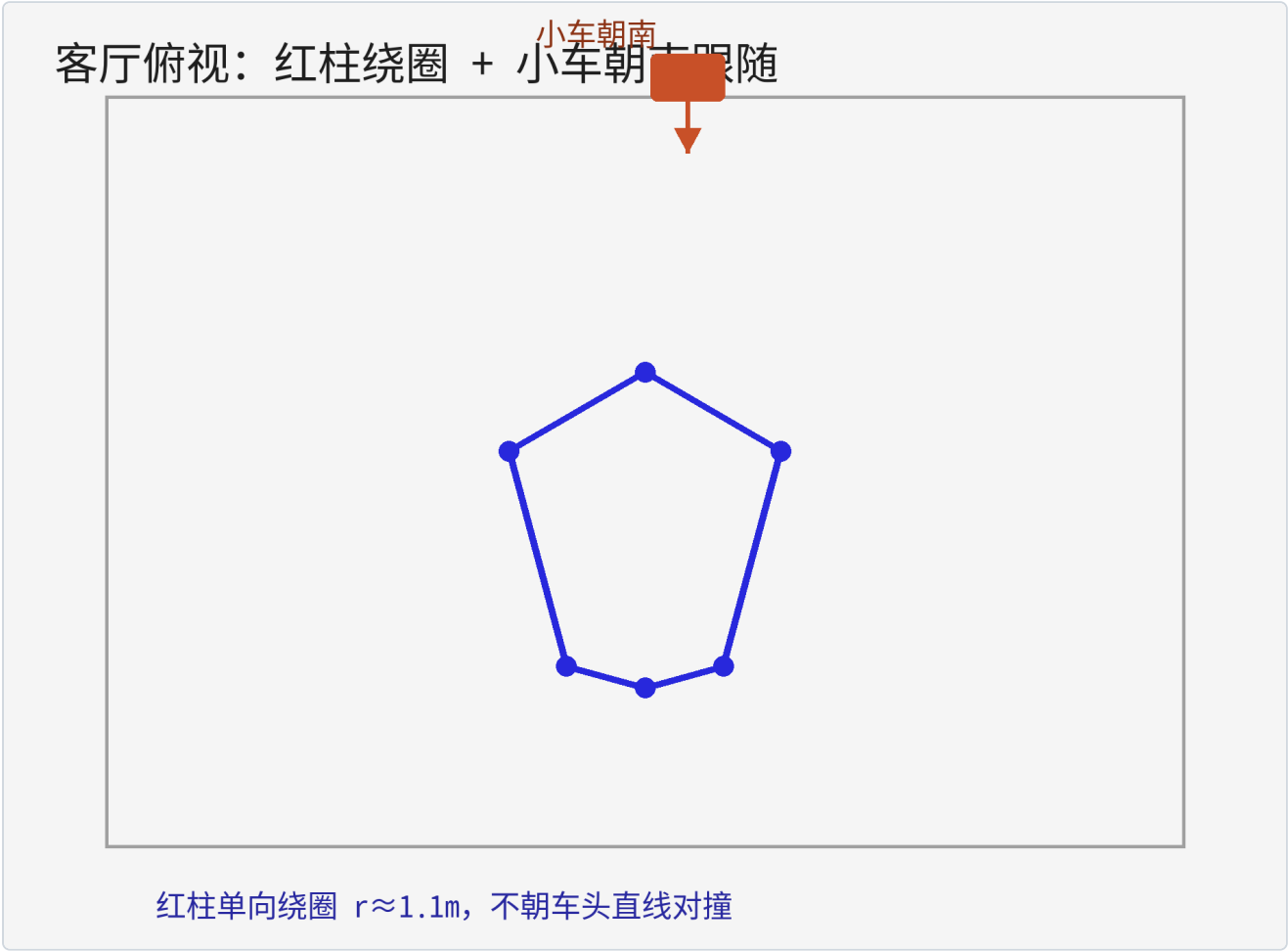
2.5 Gazebo 演示：红柱绕圈（已定型）

lite 适合验方向；给学员看车真的在跟用 Gazebo：

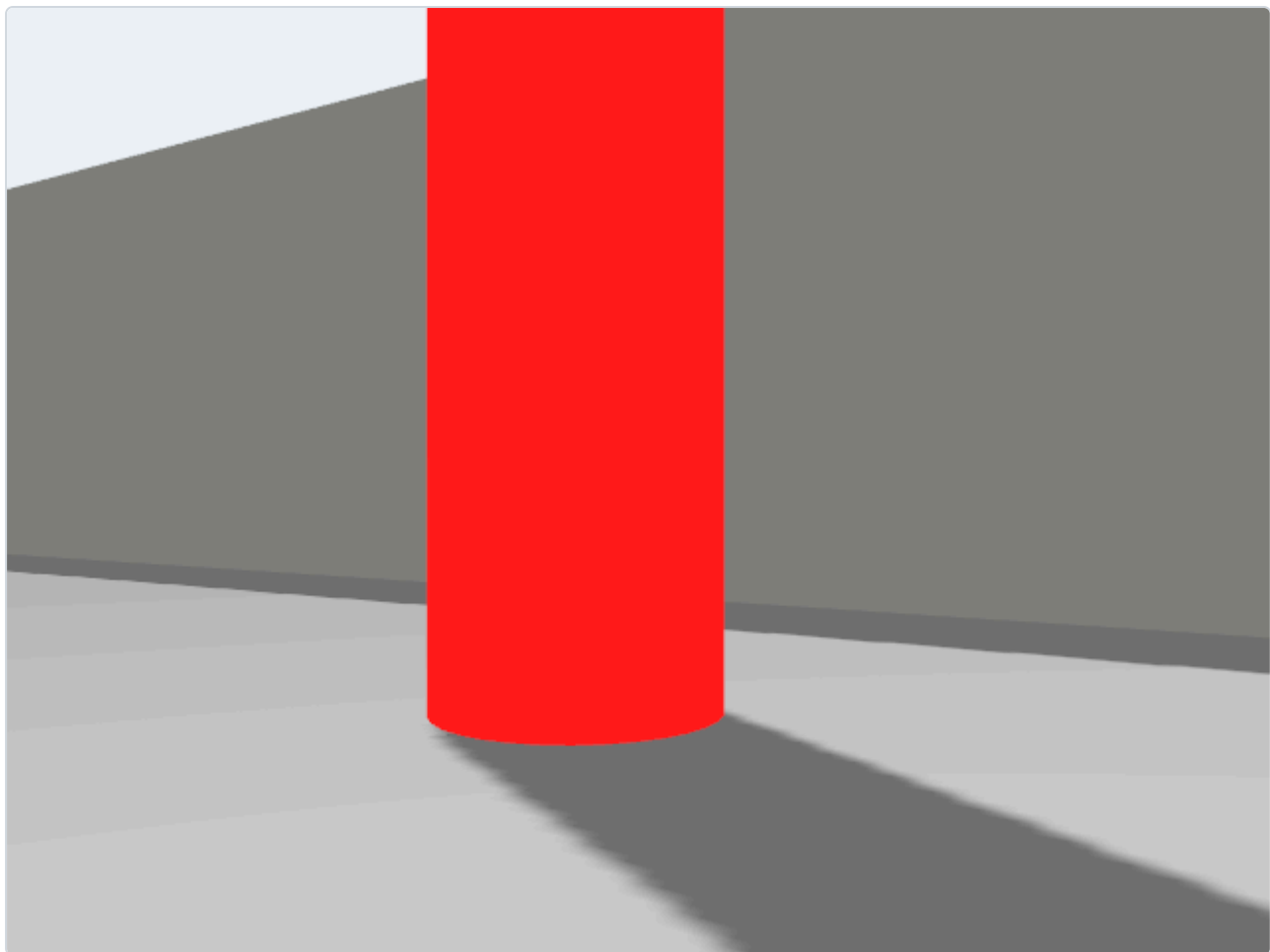
源码：

- 驱动：src/mecamind_tools/mecamind_tools/follow_target_mover.py（_on_timer / _set_pose）
- 世界：src/mecamind_gazebo/worlds/three_room_house_follow.sdf
- 总装参数：src/mecamind_bringup/launch/follow.launch.py 里 follow_target_mover / follow_controller 参数块

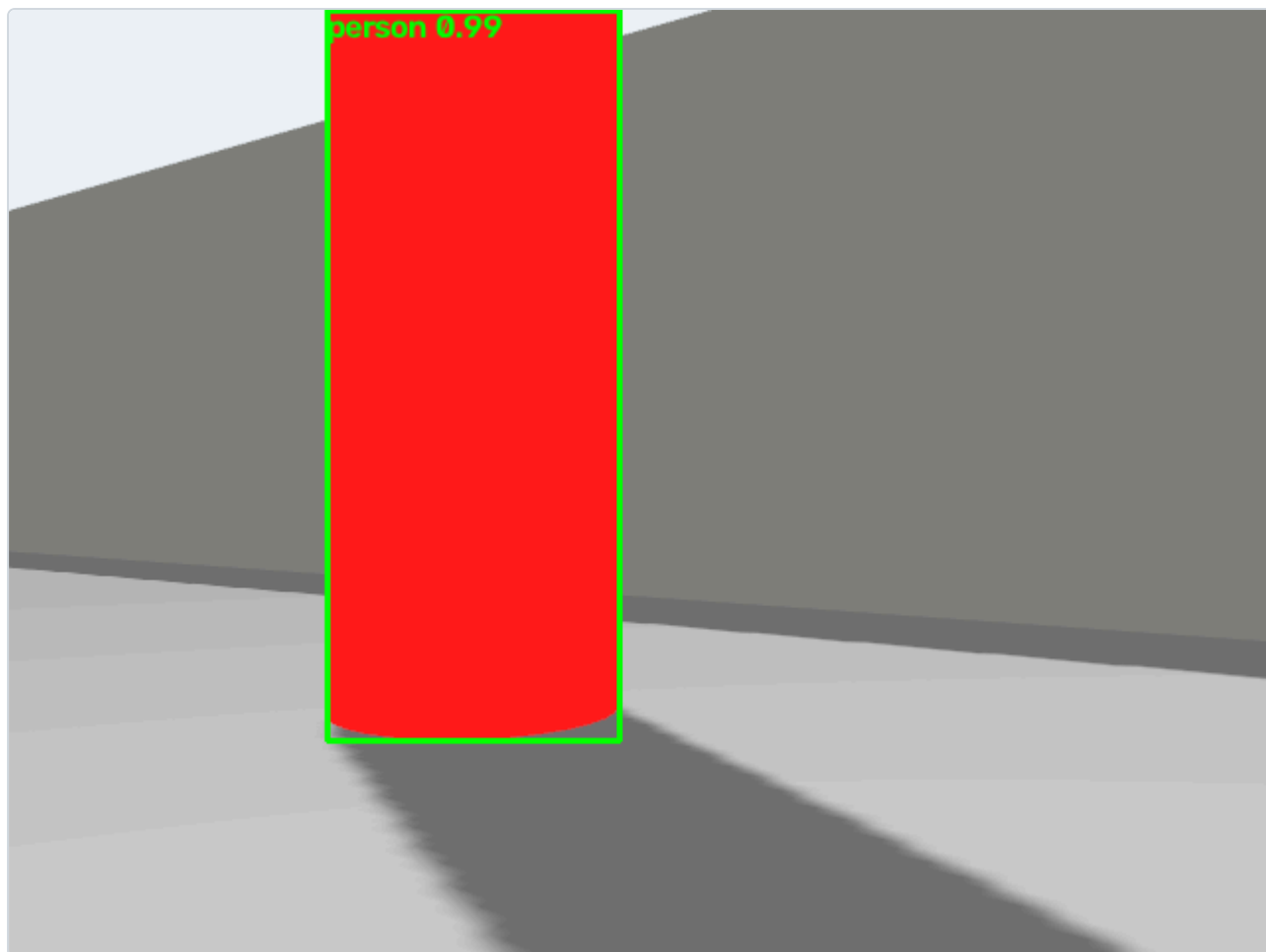
项	取值	原因
世界	three_room_house_follow.sdf	出生位姿专为跟随
检测	相机 + HSV	红柱不是 COCO person
轨迹	南侧空地绕圈， $r \approx 1.1$ m	活动范围够看，不对撞车头
小车	$(-2.0, 1.95)$ 朝南	旁观跟随
desired_width	0.36	跟近一截，过近仍退



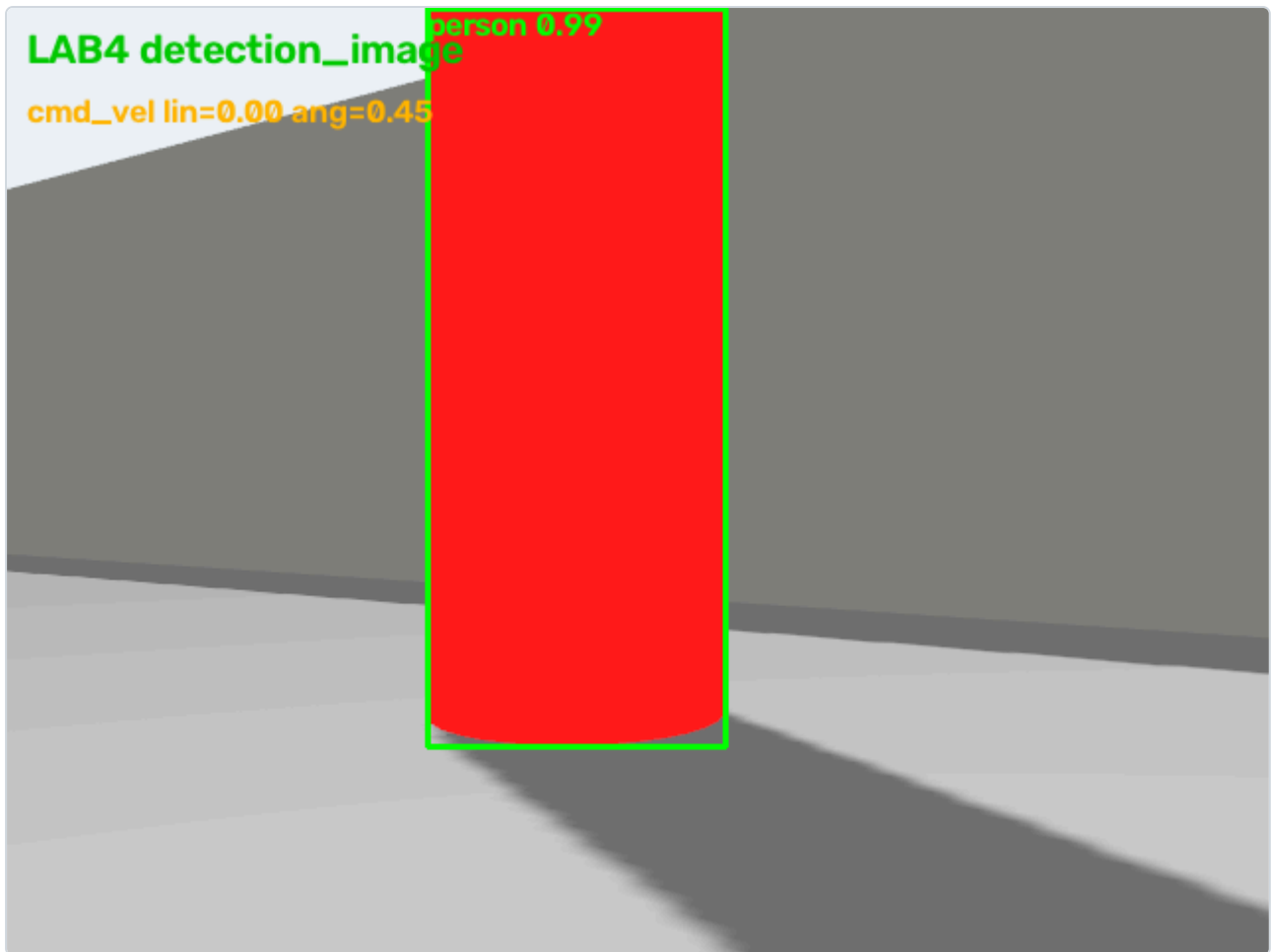
俯视：小车朝南，红柱南侧绕圈



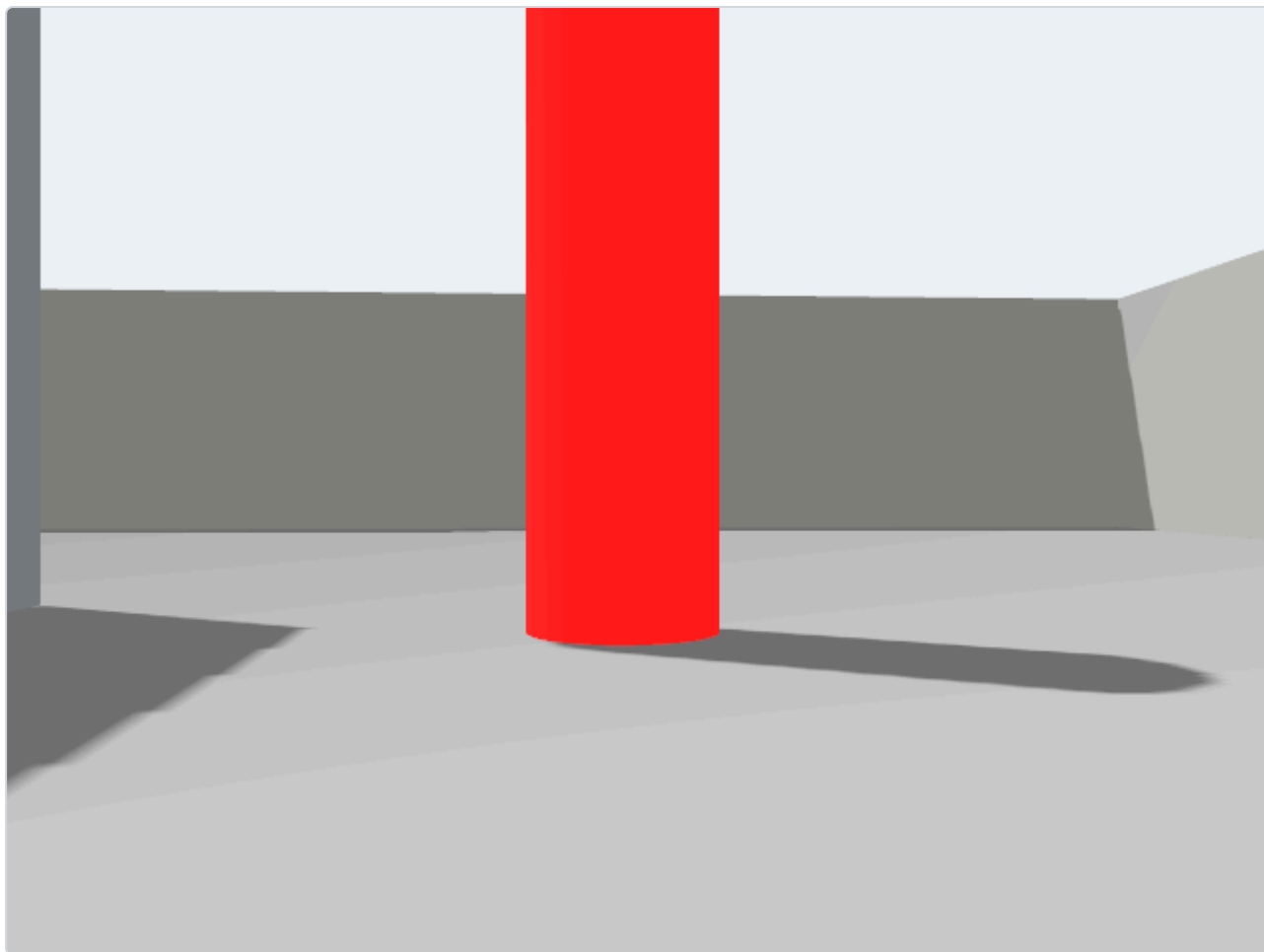
车载相机看到的红柱



HSV 检测调试图

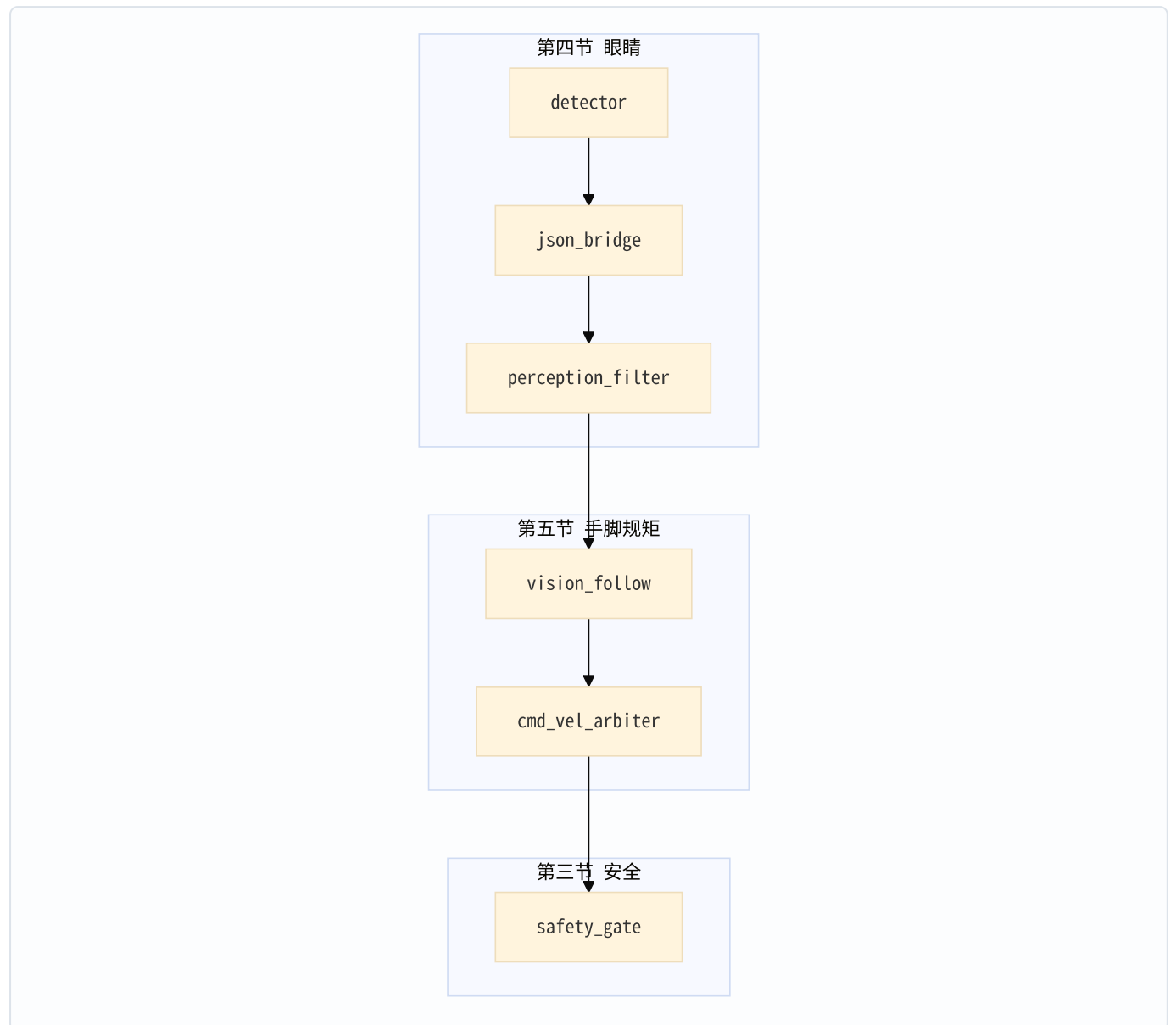


LAB4: 检测框 + 当时 cmd_vel



近距离跟随参考

2.6 与第三、四节如何拼接



简历可写：「多源速度仲裁下的视觉伺服跟随（归一化 P 控制），含最新帧推理管线与急停锁存。」

2.7 速查

关键文件

路径	角色
follow.launch.py	统一入口
detector_node.py	多线程 + LatestFrameQueue
perception_filter.py	连续帧确认
vision_follow_controller.py	P 跟随
cmd_vel_arbiter.py	速度仲裁
follow_target_mover.py	红柱绕圈
three_room_house_follow.sdf	跟随世界
scripts/test_cp5.sh	CP5

关键话题 / 参数

话题或参数	含义
/mecamind/follow_enable	跟随总开关
/mecamind/estop	急停锁存
/cmd_vel_follow → /cmd_vel	意图到执行
desired_width	越大跟越近
source_timeout_sec	仲裁新鲜度

3. 启动前准备

```
cd ~/mecamind_ros2
source /opt/ros/jazzy/setup.bash
bash scripts/cleanup_ros2.sh
colcon build --packages-select \
  mecamind_interfaces mecamind_perception mecamind_tools \
  mecamind_bringup mecamind_gazebo --symlink-install
source install/setup.bash
ros2 launch mecamind_bringup follow.launch.py --show-args
```

4. LAB 1: lite + synthetic 冒烟

目标：不靠 Gazebo/YOLO，先验证「目标偏右 → 角速度为负」。

```
ros2 launch mecamind_bringup follow.launch.py \
  backend:=lite source:=synthetic infer_backend:=hsv use_gui:=false
```

```
ros2 topic pub --once /mecamind/follow_enable std_msgs/msg/Bool '{data: true}'
ros2 topic echo /cmd_vel_follow
```

过关：使能后有非零跟随速度；目标偏右时 `angular.z < 0`。

5. LAB 2：关闭跟随应停车

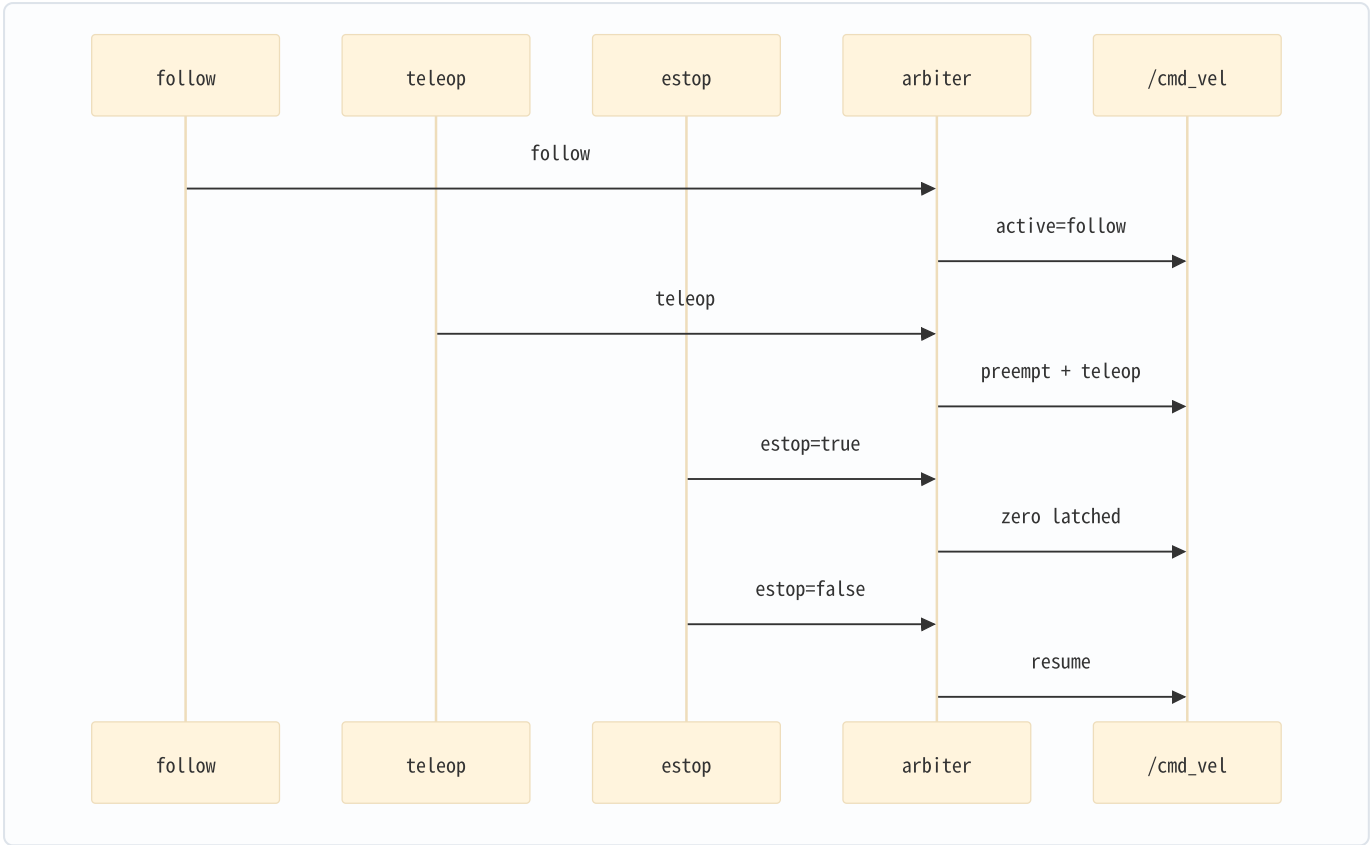
```
ros2 topic pub --once /mecamind/follow_enable std_msgs/msg/Bool '{data: false}'
ros2 topic echo /cmd_vel --once
```

过关：关闭后短时间内跟随不再驱动前进。

6. LAB 3：遥控抢占与急停

```
ros2 topic echo /mecamind/arbiter_state
ros2 topic pub /cmd_vel_teleop geometry_msgs/msg/Twist \
  '{linear: {x: 0.15}, angular: {z: 0.0}}' -r 20
# Ctrl+C 后等待超时，应回到 follow

ros2 topic pub --once /mecamind/estop std_msgs/msg/Bool '{data: true}'
ros2 topic pub --once /mecamind/estop std_msgs/msg/Bool '{data: false}'
```



过关：遥控时 `active=teleop`；急停期间速度为零。

7. LAB 4: Gazebo 红柱绕圈（主演示）

```
bash scripts/cleanup_ros2.sh
ros2 launch mecamind_bringup follow.launch.py \
  backend:=gazebo use_gui:=true \
  source:=camera infer_backend:=hsv \
  moving_target:=true target_label:=person
```

可选调试图： `python3 scripts/show_detection_image.py`

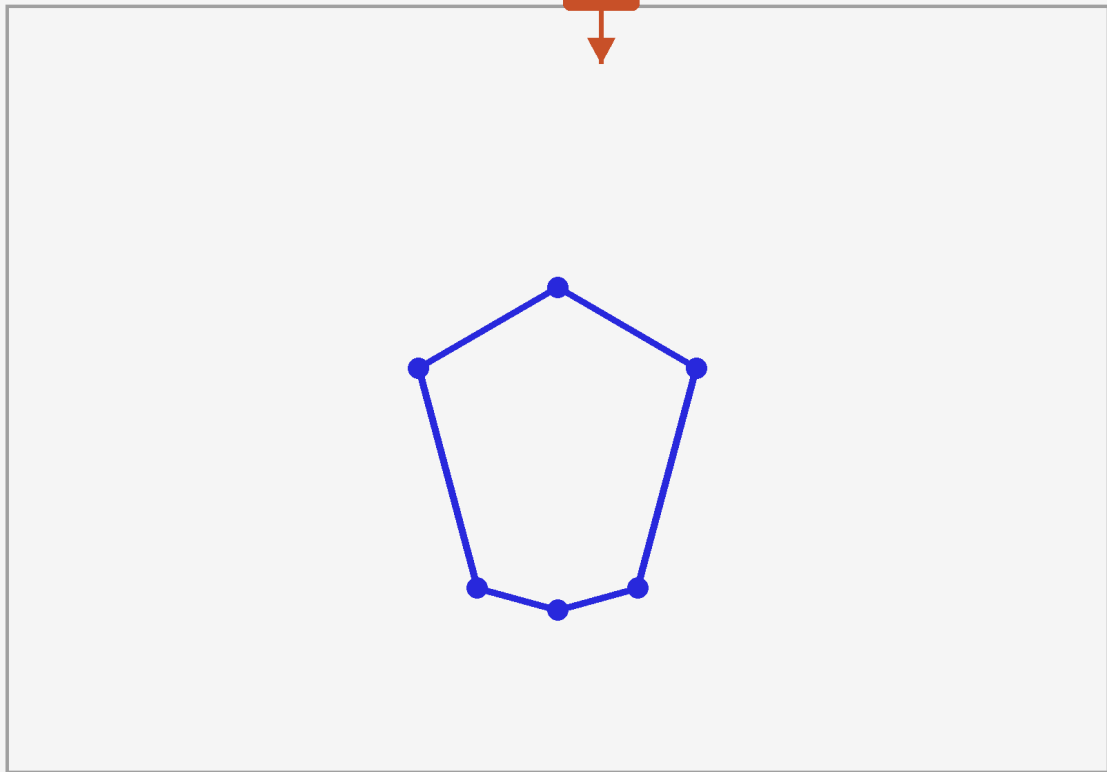
时间	现象
0~6 s	仿真起来、出生位姿确认
~10 s	自动使能，红柱开绕
之后	绿框套住红柱，车转向+跟随

热更新距离：

```
ros2 param set /mecamind_vision_follow_controller desired_width 0.40 # 更近
ros2 param set /mecamind_vision_follow_controller desired_width 0.30 # 更远
```

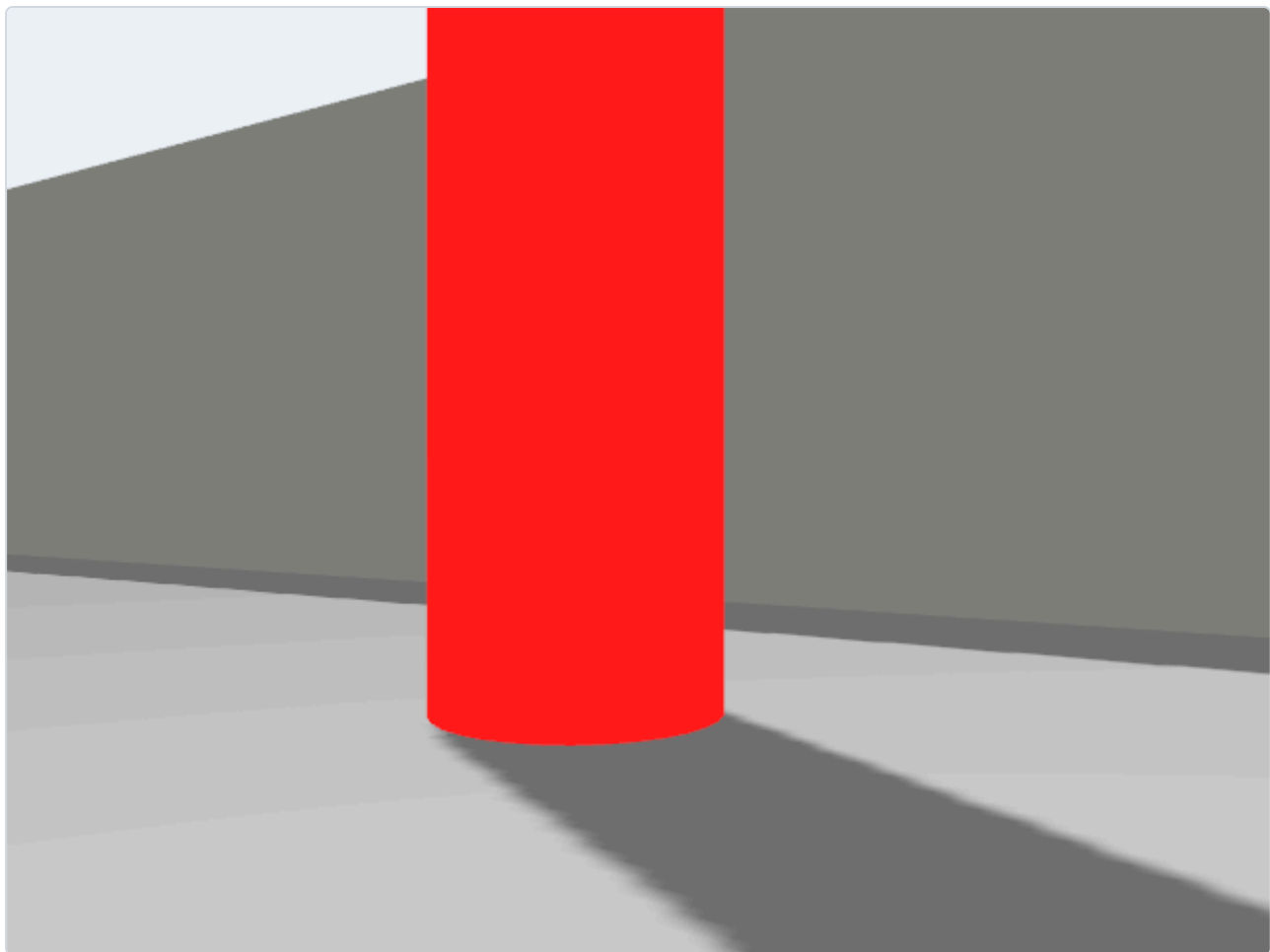
对照本节插图（做完应能对上号）：

客厅俯视：红柱绕圈 + 小车朝南跟随

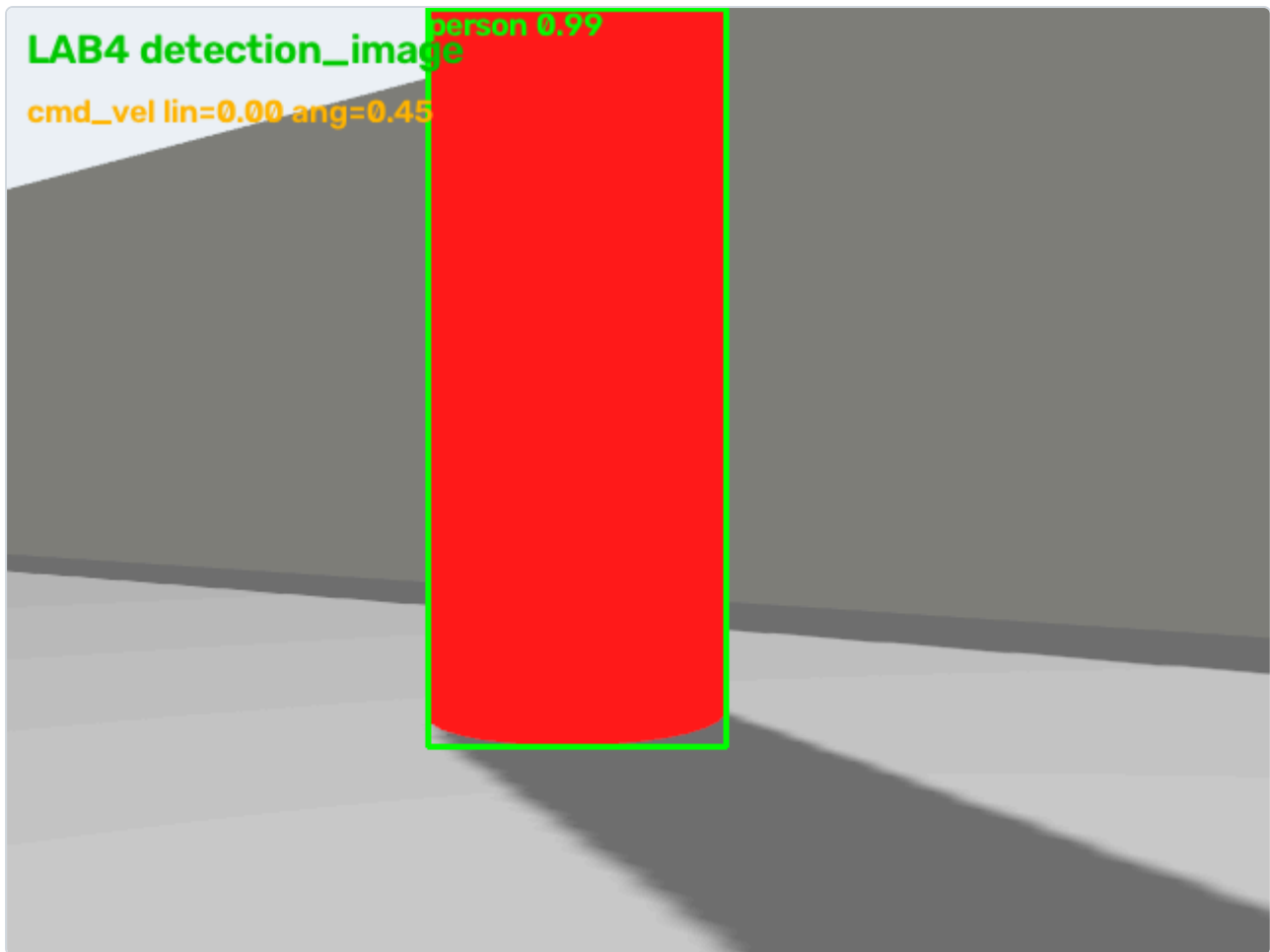


红柱单向绕圈 $r \approx 1.1\text{m}$ ，不朝车头直线对撞

俯视绕圈



车载相机



检测 HUD (绿框 + cmd_vel)

过关：日志有「绕圈」与 Vision follow enabled；检测稳定；红柱不对撞车头。

8. CP5 自动验收

```
bash scripts/cleanup_ros2.sh
bash scripts/test_cp5.sh
```

成功：MECAMIND_CP5_OK，报告 maps/mecamind_cp5_report.json。

检查点：角速度方向、关闭跟随停车、遥控抢占、急停锁存。

可选：MECAMIND_CP5_BACKEND=gazebo bash scripts/test_cp5.sh

9. 常见问题

Q1: 红柱不动/乱跳

多个 `gz sim` 残留 → `bash scripts/cleanup_ros2.sh`。

Q2: YOLO 检不到红柱

红柱不是 COCO person；演示用 HSV。

Q3: follow_enable 发一次无效

连发几次，或等自动 10 s。

Q4: 跟太近穿模

减小 `desired_width`；确认后退与 `/scan` 正常。

Q5: dropped_frames 上涨

实时跟随里通常健康（丢旧帧保延迟）。

Q6: 遥控回不了跟随

遥控节点仍在刷 0 速 → 仲裁认为 teleop 仍新鲜。

Q7: CP5 锁冲突

清理 `/tmp/mecamind_cp5.lock.d`（确认无旧进程）。

10. 验收清单

- ☐ 能指着总框架图讲四问
 - ☐ 能说清「用 P 控制、不用 DeepSORT/卡尔曼」的原因
 - ☐ 理解队列容量 1 与 `desired_width`
 - ☐ LAB1~4 过关
 - ☐ `MECAMIND_CP5_OK`
-

11. 课后作业

1. `desired_width` 取 0.28 / 0.36 / 0.42，各跟 30 秒并截图对比。
 2. hsv vs onnx (若已装) 对比 `inference_ms` 与 `dropped_frames`。
 3. 自绘时序：跟随→遥控→松手→急停→解除，标 `arbiter_state`。
 4. (选做) 开关 `search_on_loss`，遮挡目标 3 秒对比行为。
-

12. 下一课预告：10.3.6

语音/大模型意图将接到同一套开关上：`follow_enable`、`estop`、`arbiter_state`，形成导航/巡逻/跟随/停止的完整作品集闭环。

附录 A：配图与白板

课堂白板 (可在 Excalidraw 里继续涂改)：

- `docs/10.3.5_多线程推理与视觉跟随控制.excalidraw`
- 预览图：`docs/10.3.5_多线程推理与视觉跟随控制.png`

工程走读 (源码位置详解)：`docs/10.3.5_工程版源码走读.md`

文件（均在 images/10.3.5/）	用途
framework_overview_cn.png	总框架
topic_pipeline_cn.png	话题链
latest_frame_queue.png	多线程队列
principle_simple_vs_complex.png	原理：用什么/不用什么
follow_law_board.png	控制律板书
follow_p_control.png	cx 误差示意
arbiter_priority.png	仲裁优先级
arbiter_state_card.png	状态截取
follow_loop_topview.png	绕圈俯视
follow_closed_loop_cn.png	跟随闭环：看→算→动
gazebo_follow_cam.png	相机实拍
detection_follow_hsv.png	检测框
lab4_detection_hud.png	LAB4 HUD
gazebo_follow_close.png	近距参考

附录 B：一页纸命令

```
bash scripts/cleanup_ros2.sh

# 零依赖
ros2 launch mecamind_bringup follow.launch.py backend:=lite source:=synthetic infer_backend:=hsv

# Gazebo 演示
ros2 launch mecamind_bringup follow.launch.py \
  backend:=gazebo use_gui:=true source:=camera \
  infer_backend:=hsv moving_target:=true target_label:=person

ros2 topic pub --once /mecamind/follow_enable std_msgs/msg/Bool '{data: true}'
ros2 topic pub --once /mecamind/estop std_msgs/msg/Bool '{data: true}'
ros2 topic pub --once /mecamind/estop std_msgs/msg/Bool '{data: false}'

bash scripts/test_cp5.sh
```

文档版本：对齐当前 `follow.launch.py` / CP5（红柱南侧绕圈、`desired_width=0.36`，跟踪原理为归一化 P 控制）。